

Máy tự động hậu tố

Trần Lập Kiệt, Trường Ngoại ngữ Hàng Châu

Nguồn gốc: Suffix Automaton

1 Mở đầu

Tài liệu này giới thiệu **máy tự động hậu tố** (suffix automaton, viết tắt là SAM) và một số ứng dụng kinh điển trong xử lý chuỗi. Mục tiêu là có một cấu trúc tuyến tính theo độ dài chuỗi nhưng vẫn trả lời được nhiều câu hỏi về chuỗi con, vị trí xuất hiện và thứ tự từ điển.

Trước hết xét bài SPOJ 1812, **Longest Common Substring II**: cho $N \leq 10$ chuỗi, mỗi chuỗi có độ dài không quá 100000, hãy tìm chuỗi con liên tiếp chung dài nhất của tất cả các chuỗi. Một cách đơn giản là nhị phân độ dài rồi dùng băm, đạt $O(L \log L)$ với L là tổng độ dài. Cách này quen thuộc nhưng trên môi trường chậm như SPOJ vẫn dễ bị quá thời gian. SAM đưa ra một hướng khác, tuyến tính hơn và phù hợp với các bài trực tuyến.

2 Các công cụ xử lý chuỗi trong OI

Trong OI, các công cụ xử lý chuỗi thường gặp gồm:

- mảng hậu tố (suffix array);
- cây hậu tố (suffix tree);
- máy tự động Aho–Corasick;
- băm chuỗi;
- máy tự động hậu tố.

SAM có thể xem là một phiên bản nén của cây trie các hậu tố, nhưng số trạng thái và số cạnh đều chỉ tuyến tính. So với cây hậu tố, thuật toán xây dựng của SAM ngắn và dễ cài đặt hơn, dù các bất biến ban đầu có thể khó nhìn.

3 Máy tự động hữu hạn

Một máy tự động hữu hạn dùng để nhận dạng chuỗi. Với máy tự động A , nếu A nhận chuỗi S thì viết $A(S) = \text{True}$, ngược lại là False .

Một máy tự động gồm năm phần:

- Σ : bảng chữ cái;
- state: tập trạng thái;
- init: trạng thái ban đầu;

- end: tập trạng thái kết thúc;
- trans: hàm chuyển trạng thái.

Ký hiệu $\text{trans}(s, c)$ là trạng thái đi đến khi đang ở trạng thái s và đọc ký tự c . Nếu chuyển này không tồn tại, ta xem kết quả là null; trạng thái null chỉ chuyển về chính nó. Mở rộng cho xâu t , $\text{trans}(s, t)$ là trạng thái thu được sau khi lần lượt đọc từng ký tự của t :

```
cur = s
for i = 0 .. length(t)-1:
    cur = trans(cur, t[i])
```

4 Định nghĩa SAM

Cho xâu mẹ S . Máy tự động hậu tố của S là một máy tự động nhận đúng tất cả các hậu tố của S :

$$\text{SAM}(x) = \text{True} \iff x \text{ là hậu tố của } S.$$

Sau khi xây dựng xong, cùng cấu trúc này cũng nhận biết được tất cả xâu con của S . Ký hiệu

$$\text{ST}(t) = \text{trans}(\text{init}, t)$$

là trạng thái đạt được khi đọc t từ gốc. Khi $\text{ST}(t) \neq \text{null}$, t là một xâu con của S .

Cách thô sơ nhất là đưa mọi hậu tố của S vào một trie. Khi đó gốc là trạng thái ban đầu, cạnh trie là hàm chuyển, còn các lá là trạng thái kết thúc. Tuy nhiên trie này có thể có kích thước $O(n^2)$. SAM tối thiểu hóa các trạng thái có cùng hành vi, nhờ đó chỉ còn $O(n)$ trạng thái.

5 Tập vị trí kết thúc và trạng thái

Với một xâu con t của S , đặt

$$\text{Right}(t) = \{i \mid S[i - |t| + 1..i] = t\},$$

tức là tập mọi vị trí kết thúc của những lần xuất hiện của t trong S .

Trong SAM tối thiểu, một trạng thái đại diện cho một lớp các xâu con có cùng tập Right. Nếu trạng thái là s , ta viết tập này là $\text{Right}(s)$. Các độ dài của xâu con thuộc cùng trạng thái tạo thành một đoạn liên tiếp:

$$[\text{Min}(s), \text{Max}(s)].$$

Một cặp gồm tập Right và độ dài xác định duy nhất một xâu con.

Với mỗi trạng thái s khác gốc, $\text{parent}(s)$ là trạng thái x sao cho $\text{Right}(s)$ là tập con thực sự của $\text{Right}(x)$, và $|\text{Right}(x)|$ nhỏ nhất có thể. Các liên kết parent tạo thành một cây, thường gọi là **cây parent** hay **suffix-link tree**. Ta có bất biến quan trọng:

$$\text{Max}(\text{parent}(s)) = \text{Min}(s) - 1.$$

6 Tính chất về xâu con

Mọi xâu con của S đều nằm trong một trạng thái nào đó của SAM. Do đó:

$$t \text{ là xâu con của } S \iff \text{ST}(t) \neq \text{null}.$$

Nếu đã đi đến trạng thái chứa t , số lần xuất hiện của t chính là $|\text{Right}(t)|$, tức kích thước của tập Right tương ứng.

Không nên lưu trực tiếp toàn bộ tập Right ở mỗi trạng thái vì tốn bộ nhớ. Ta dùng cây parent : tập Right của một trạng thái là hợp các tập Right của những lá trong cây con của nó. Nếu đánh số theo thứ tự DFS của cây parent , toàn bộ một cây con là một đoạn liên tiếp; nhờ đó có thể dùng các cấu trúc dữ liệu trên đoạn để tìm nhanh mọi vị trí xuất hiện của một xâu con.

7 Xây dựng tuyến tính

Thuật toán xây dựng SAM là trực tuyến: đọc xâu từ trái sang phải và thêm từng ký tự. Dù ý tưởng của nó không hiển nhiên, phần cài đặt ngắn hơn nhiều so với cây hậu tố.

Giả sử SAM hiện tại biểu diễn xâu S có độ dài L , trạng thái last đại diện cho toàn bộ xâu S . Khi thêm ký tự c , ta tạo trạng thái mới np với

$$\text{Max}(np) = L + 1.$$

Trạng thái này đại diện cho các hậu tố mới kết thúc ở vị trí $L + 1$.

Từ $p = \text{last}$, đi ngược theo các liên kết parent . Với mọi trạng thái p chưa có cạnh c , thêm cạnh $p \xrightarrow{c} np$. Khi quá trình dừng có ba trường hợp.

1. Không còn trạng thái nào, tức đã đi qua gốc. Khi đó $\text{parent}(np) = \text{init}$.
2. Tồn tại cạnh $p \xrightarrow{c} q$ và $\text{Max}(p) + 1 = \text{Max}(q)$. Khi đó q đã là trạng thái đúng để nối tiếp, đặt $\text{parent}(np) = q$.
3. Tồn tại cạnh $p \xrightarrow{c} q$ nhưng $\text{Max}(p) + 1 < \text{Max}(q)$. Khi đó cần tách q . Tạo trạng thái sao chép nq , sao chép toàn bộ chuyển trạng thái của q , đặt $\text{Max}(nq) = \text{Max}(p) + 1$ và $\text{parent}(nq) = \text{parent}(q)$ cũ. Sau đó đặt $\text{parent}(q) = \text{parent}(np) = nq$, rồi đổi mọi cạnh $v \xrightarrow{c} q$ trên đường parent liên quan thành $v \xrightarrow{c} nq$.

Cuối cùng đặt $\text{last} = np$. Trong bước tách, các chuyển của nq giống q vì khi đang xử lý ký tự mới ở vị trí $L + 1$, phần sau trạng thái này không phụ thuộc riêng vào vị trí mới.

8 Mã giả

```

extend(c):
    np = new_state()
    maxlen[np] = maxlen[last] + 1
    p = last

    while p exists and next[p][c] is absent:
        next[p][c] = np
        p = parent[p]

    if p does not exist:
        parent[np] = root
    else:
        q = next[p][c]
        if maxlen[p] + 1 == maxlen[q]:
            parent[np] = q

```

```

else:
    nq = clone_state(q)
    maxlen[nq] = maxlen[p] + 1
    parent[nq] = parent[q]
    parent[q] = nq
    parent[np] = nq
    while p exists and next[p][c] == q:
        next[p][c] = nq
        p = parent[p]

last = np

```

Mỗi lần thêm ký tự chỉ tạo nhiều nhất hai trạng thái. Vì vậy số trạng thái của SAM không vượt quá $2n - 1$ với $n > 0$. Số cạnh cũng tuyến tính; nhìn trực quan, các cạnh được thêm và đổi hướng dọc theo các suffix link, tổng số lần thao tác trên toàn bộ quá trình là tuyến tính.

9 I. Xâu quay vòng nhỏ nhất

Cho xâu S . Mỗi lần được chuyển ký tự đầu tiên xuống cuối xâu. Hãy tìm xâu có thứ tự từ điển nhỏ nhất có thể thu được. Ví dụ với BBAAB, kết quả là AABBB.

Xét xâu SS . Bài toán trở thành tìm xâu con độ dài $|S|$ nhỏ nhất theo thứ tự từ điển của SS . Xây SAM cho SS ; từ trạng thái gốc, mỗi lần đi theo cạnh có nhãn nhỏ nhất, đi đúng $|S|$ bước thì thu được đáp án. Lý do là mọi phép quay của S đều là một xâu con độ dài $|S|$ trong SS , và đi tham lam theo ký tự nhỏ nhất sinh ra xâu nhỏ nhất trong ngôn ngữ được SAM biểu diễn.

10 II. SPOJ NSUBSTR

Cho xâu S . Gọi $F(x)$ là số lần xuất hiện lớn nhất trong tất cả xâu con độ dài x của S . Cần tính $F(1), F(2), \dots, F(|S|)$, với $|S| \leq 250000$.

Xây SAM của S . Với mỗi trạng thái s , các xâu mà nó đại diện có độ dài nằm trong đoạn $[\text{Min}(s), \text{Max}(s)]$, và cùng có số lần xuất hiện $|\text{Right}(s)|$. Vì thế dùng $|\text{Right}(s)|$ để cập nhật $F(\text{Max}(s))$. Sau đó duyệt i giảm dần và gán

$$F(i - 1) = \max(F(i - 1), F(i)).$$

Việc lan truyền này đúng vì nếu một xâu dài xuất hiện nhiều lần thì tiền tố ngắn hơn của nó cũng xuất hiện ít nhất bấy nhiêu lần.

11 III. BZOJ2555 SubString

Bài toán yêu cầu duy trì một xâu, hỗ trợ hai thao tác trực tuyến:

- thêm một ký tự vào cuối xâu;
- hỏi một xâu xuất hiện bao nhiêu lần trong xâu hiện tại.

Ta xây SAM động bằng thủ tục thêm ký tự ở trên. Điều thực sự ảnh hưởng đến đáp án là kích thước tập Right của trạng thái truy vấn. Trong mỗi bước xây dựng, liên kết parent chỉ thay đổi hằng số lần;

trạng thái mới np tương ứng với một lần xuất hiện mới, nên mọi tổ tiên của np trên cây parent cần tăng kích thước Right thêm 1.

Có hai hướng duy trì:

- dùng cây động để duy trì cây parent;
- dùng cây cân bằng để duy trì thứ tự DFS của cây parent.

Các kỹ thuật này nằm ngoài trọng tâm của bài giảng, nên tài liệu gốc chỉ nêu ý tưởng.

12 IV. SPOJ SUBLEX

Cho một xâu S độ dài không quá 90000. Mỗi truy vấn hỏi xâu thứ K theo thứ tự từ điển trong tập các xâu con phân biệt của S ; số truy vấn không quá 500.

Xây SAM của S . Với mỗi trạng thái, tiền xử lý số lượng xâu con phân biệt có thể đi tới từ trạng thái đó. Sau đó trả lời truy vấn bằng cách đi từ gốc theo thứ tự nhãn cạnh tăng dần: nếu toàn bộ nhánh hiện tại chứa ít hơn K xâu thì bỏ qua nhánh đó và trừ vào K ; ngược lại chọn cạnh đó, in thêm ký tự cạnh, rồi tiếp tục. Đây là cách DFS theo thứ tự từ điển trên DAG của SAM.

13 V. SPOJ LCS

Cho hai xâu A và B , mỗi xâu có độ dài nhỏ hơn 100000, hãy tìm xâu con liên tiếp chung dài nhất. Xây SAM cho A .

Duyệt các ký tự của B bằng SAM của A . Duy trì trạng thái hiện tại s và độ dài khớp hiện tại ℓ .

- Nếu có cạnh nhãn c từ s , đi theo cạnh đó và tăng ℓ .
- Nếu không có, đi ngược theo parent cho đến khi gặp trạng thái a có cạnh nhãn c . Khi đó chuyển đến $\text{trans}(a, c)$ và đặt $\ell = \text{Max}(a) + 1$.
- Nếu không có tổ tiên nào có cạnh nhãn c , quay về gốc và đặt $\ell = 0$.

Trong quá trình duyệt, cập nhật độ dài khớp lớn nhất đạt được ở từng trạng thái. Vì kết quả của một trạng thái cũng có thể đóng góp cho parent của nó, cuối cùng duyệt các trạng thái theo Max giảm dần để đẩy kết quả lên cây parent. Đáp án là giá trị lớn nhất thu được.

14 VI. SPOJ LCS2

Bài LCS2 mở rộng bài trước cho nhiều xâu. Xây SAM của xâu đầu tiên. Với mỗi xâu còn lại, chạy quy trình khớp như trong SPOJ LCS để biết tại mỗi trạng thái độ dài lớn nhất mà xâu này có thể khớp. Sau khi xử lý một xâu, lan truyền kết quả lên cây parent, rồi lấy minimum theo từng trạng thái qua tất cả các xâu. Giá trị lớn nhất còn lại chính là độ dài xâu con chung dài nhất của toàn bộ tập xâu.

15 Một vài cấu trúc liên quan

Ngoài suffix automaton còn có nhiều biến thể và họ hàng:

- factor automaton;
- suffix oracle;

- factor oracle;
- suffix cactus.

Factor oracle là một máy tự động có thể nhận tất cả xâu con của xâu đã cho, nhưng cũng có thể nhận thêm một số xâu không thuộc tập đó. Vì vậy chữ “oracle” ở đây đúng nghĩa là một lời dự đoán: nhanh và đơn giản, nhưng không nhất thiết chính xác tuyệt đối. Tài liệu gốc chỉ nhắc tới nó như phần mở rộng, không đi sâu vào ứng dụng trong OI.

16 Ghi chú về bản dịch

Một số trang trong PDF nguồn chứa hình minh họa hoặc mã được nhúng dưới dạng đối tượng đồ họa nên không trích xuất được văn bản bằng công cụ PDF thông thường. Các phần đó được dựng lại theo đúng trình tự bài giảng và theo thuật toán SAM chuẩn: định nghĩa Right, cây parent, thuật toán thêm ký tự, và các ứng dụng nêu trong slide.